

zero_shortcomings_audit

scripts/

zero_shortcomings_audit.sh

Revision: 8 **Last modified:** 2026-09-26T17:35:00Z

Purpose: Unified enumeration, closure-evidence re-verification, and standing-check entry point across this project's three tracked “unfinished/gap/shortcoming” surfaces (open backlog items, unimplemented named gates, open coverage-escapes). See specs/003-zero-shortcomings-audit/contracts/cli.md for the full CLI contract.

Usage: scripts/zero_shortcomings_audit.sh <enumerate|verify-closure|standing-check> [options]

-h / --help prints the usage text and exits 0. Running with no mode prints the usage and exits 4. An unknown mode prints unknown mode: <x> plus the usage and exits 4. **Every usage error in every mode exits 4**, so a usage refusal can never be mistaken for a verification mismatch (exit 1). (The built-in --help text lists every mode and option; this guide gives the full semantics.)

Modes

enumerate [--json] [--surface S] [--sort-by-risk]

Read-only. S is one of all (default), backlog, gates, escapes, blocked.

- Default text output: Open backlog items, Unimplemented gates, Open coverage-escapes counts (only for the selected surface).
- --json: one JSON object with the keys backlog_open, gates_unimplemented, escapes_open (only the selected surfaces' keys).

- `--surface blocked`: lists every Operator-blocked item as `id|unblock_condition` (left-joined from `operator_block_details`, ordered by `id`), never a bare count. An Operator-blocked item with **no** details row, or with a blank condition (only spaces, tabs, newlines or carriage returns), is listed as `id|MISSING-UNBLOCK-CONDITION` (never dropped). Every item is listed **once**: when it has more than one details row (a legacy table without the primary key, or the item stored twice in `items`) its distinct non-blank conditions are joined with `;`, and a line break inside a condition becomes a space. A `WARN: N Operator-blocked item(s) lack an unblock condition` line goes to `stderr`; the exit status stays 0 (enumerate reports findings, it does not gate on them).
- `--sort-by-risk (FR-012)`: prints the open backlog item ids ordered by reopen count descending (count of Reopened events in `item_history`), then `last_modified` descending, then severity (case-insensitive: `critical`, then `high/major/important`, then `medium`, then `low/minor`, then anything else or empty), then `id`. With `--json` it prints a JSON array of ids. **Only valid with `--surface backlog`.**
- Enumeration failure (exit 1, message on `stderr`): the tracker DB (backlog/blocked surfaces) is missing or unreadable (tracker DB missing or unreadable: `<path>` – checked before any `sqlite3` call, so a missing DB is never created as an empty file).
- Usage errors (exit 4, message on `stderr`): `--surface` given with no value (enumerate: `--surface` requires a value ...); unknown option (enumerate: unknown option: `X`); unrecognized `--surface` value (enumerate: unrecognized `--surface` value: `'X'` (expected `all|backlog|gates|escapes|blocked`)); `--sort-by-risk` with any surface other than backlog (enumerate: `--sort-by-risk` applies only to `--surface backlog` (got `'X'`)).
- The gate count fails loud (exit 1, message naming the gate script) when the gate-ledger script prints no parseable `unimplemented=N` line, rather than reporting 0.

`verify-closure <item-id> [--reopen-on-mismatch] [--require-layer L]`

Independently re-runs the recorded closure evidence for one item. Evidence is read from the lexically last `closure_evidence_*.md` directly inside `<AUDIT_QA_ROOT>/<item-id>/` (a deterministic choice when more than one exists). `<item-id>` must match `^[A-Za-z0-9][A-Za-z0-9._-]*$` and must not contain `..`; anything else (a `/`, a leading `-` or `.`, spaces, shell metacharacters) is rejected with `invalid item id` before the `id` is used in any path. `--require-layer L` (`source|artifact|runtime`, default `runtime`) is the minimum evidence layer accepted; the evidence's

****Evidence Layer:**** field (default source when absent) must rank at or above it (source < artifact < runtime; an unrecognized label ranks as source).

Required evidence-file fields:

- ****Evidence Layer:**** (optional; see above)
- ****Test Type:**** (FR-010) – required. A comma-separated list of one or more values, each compared case-insensitively after trimming: unit, integration, e2e, security, stress, chaos, scaling, ui, challenge (e.g. ****Test Type:**** unit, integration). An unknown member, or a list with no members, is refused (exit 2). **Coverage check:** the declared list must include every type the command visibly exercises, where that is mechanically decidable – a tests/<type>/ path token in the command (for unit, integration, e2e, security, stress, chaos, scaling) or a challenges/ path token (challenge). A missing type is refused (exit 2, ... exercises undeclared test type(s): X). **Honest limit:** a command with no such token (go test ./..., pytest -k, an ad-hoc pipeline, tests/audit/, tests/load/, tests/ux/, ...) cannot be decided from its text; the run then prints FR-010 coverage for <id> is not mechanically decidable from its command; its declared type(s) ... are taken as stated and accepts the declaration.
- ****Command:**** <command> – the command to re-run. It runs as bash -c "<command>" in a **fresh process** with the working directory set to the repository root (the one this script lives in, not the caller's cwd) and stdin closed, so it does not inherit this script's set -euo pipefail, functions or non-exported variables and gives the same result wherever verify-closure is invoked from. Exported environment variables are inherited, as for any child process, **except BASH_ENV and ENV, which are removed** (bash would otherwise source that startup file before the command and could rewrite the very result being verified). The command runs under the ExecutionPolicy bounds (audit_dispatch_bounded: nice -n 19, plus ionice -c 3 when available – Principle XIII) and, when AUDIT_VERIFY_COMMAND_TIMEOUT is set, under timeout -k 5 <seconds>, whose process-group kill also reaches the command's children. The command's stderr is shown on the terminal only after passing through the redactor (see Redaction).
- ****Result Summary:**** <text> – compared exactly against the command's fresh stdout.

Trust model: the `**Command:**` is executed as arbitrary shell. Evidence files are trusted, tracked input – review them like code; never point `AUDIT_QA_ROOT` at untrusted files.

Exit codes:

Code	Meaning
0	The recorded command's fresh output equals the recorded Result Summary (reproduced its recorded evidence).
1	Mismatch only (MISMATCH -- recorded '...', got '...', values credential-redacted before printing).
2	No <code>closure_evidence_*.md</code> found; declared layer below --require-layer; no <code>**Test Type:**</code> ; invalid <code>**Test Type:**</code> ; no <code>**Command:**</code> field.
3	Mismatch and --reopen-on-mismatch was given and the tracker reopen failed (reopen FAILED for <id> -- the mismatch is NOT recorded in the tracker).
4	Usage error: missing or invalid item id, unknown option, --require-layer with no value or a value other than source/artifact/runtime, a non-integer AUDIT_VERIFY_COMMAND_TIMEOUT.
5	Internal refusal, the command is not run: a temp file for the guard could not be created (mktemp failed), the evidence snapshot failed, the FR-011 lock file could not be opened, or another verify-closure holds the lock (another verify-closure holds <lock> ...).
6	

Code	Meaning
	The recorded command exceeded AUDIT_VERIFY_COMMAND_TIMEOUT (timed out after Ns -- INCONCLUSIVE): neither a match nor a mismatch. The corruption guard still ran.

FR-011 serialization: before snapshotting, `verify-closure` takes an exclusive flock on a per-repository lock file (`$AUDIT_VERIFY_LOCK_FILE`, default `${TMPDIR:-/tmp}/zero_shortcomings_audit_verify_<cksum of repo root>.lock`, never inside the repository), waiting at most `$AUDIT_VERIFY_LOCK_WAIT` seconds (default 60) and refusing with exit 5 after that. Two concurrent closure checks would otherwise snapshot, run and then revert the same tracked tree, one undoing the other's legitimate writes. Without `flock(1)` the check runs unlocked and prints a WARN saying FR-011 is not enforced on that host.

--reopen-on-mismatch: on a mismatch, calls the `workable-items reopen CLI` (--why test-failed --who AI, evidence path = the evidence file, DB = `WORKABLE_ITEMS_DB_OVERRIDE` or `docs/workable_items.db`) and returns 1 when it succeeds. A failing reopen is reported with exit 3, never swallowed.

Evidence-corruption guard (FR-008): before running the recorded command the script snapshots (content hashes) every git-tracked file under `docs/qa/` of the repository root; afterwards any such file **modified or deleted** outside `<AUDIT_QA_ROOT>/<item-id>/` is restored (files that were already dirty before the run are restored from a byte copy of their pre-command content, others via `git checkout --`) and the incident is appended to `<AUDIT_QA_ROOT>/zero_shortcomings_audit/<run-id>.log` (or `$AUDIT_INCIDENT_LOG_DIR/<run-id>.log`) with a warning on the console. The guard never changes the exit code by itself.

Guard limits (stated, not hidden):

- The snapshot is taken immediately before and compared immediately after the command. A **legitimate** edit made by **another process** to a tracked `docs/qa/` file during that window is indistinguishable from corruption and **will be reverted**. Run `verify-closure` sequentially, never alongside other writers of `docs/qa/`.

- Paths with spaces or tabs are handled; a tracked path containing a **newline** cannot be snapshotted (`git hash-object --stdin-paths` is newline-delimited) and is skipped with a WARN naming it.
- Files the command **creates** (untracked) and changes inside the item's own evidence directory are left alone.

standing-check [--reverify N]

Recurring, advisory mode; **always exits 0** once its arguments are valid (a bad option or a non-integer `--reverify` value exits 4). Runs `enumerate --json` for each of the backlog, gates, escapes surfaces, **re-verifies closed items** (SC-005, below) and appends one line to `<AUDIT_STANDING_LOG_DIR>/<run-id>.log`:

```
<run-id> mode=standing-check status=ok
reverify=checked:C,match:M,mismatch:X
{"backlog_open":N,"gates_unimplemented":N,"escapes_open":N}
```

Closed-item re-verification (SC-005 – a reintroduced, previously fixed defect is caught by one normal run, with nobody re-triggering it): the closed items (status ending in (`→ Fixed.md`), never `Obsolete`) are ordered by risk – reopen count descending, then `last_modified` descending, then severity, then id – and the first N whose evidence file records a ****Command:**** are re-run, each as its own `verify-closure <id>` process (so its corruption guard runs and its incident log goes to `<AUDIT_STANDING_LOG_DIR>`). N is `--reverify N`, else `$AUDIT_REVERIFY_DEFAULT`, else 2; `--reverify 0` disables it and the line says `reverify=off`. The standing mode **never** passes `--reopen-on-mismatch`: it reports, it never edits the tracker. Bounds: each recorded command gets `$AUDIT_REVERIFY_ITEM_TIMEOUT` seconds (default 150, sized from the measured 130 s of the one closed item on the live tracker that records a command), an item is only started when that full timeout still fits in `$AUDIT_REVERIFY_BUDGET` seconds of re-verify time (default 210), a busy FR-011 lock is not waited for, and an outer backstop timeout guards each process. Every outcome other than a clean match makes the run `status=degraded` `reason=...reverify` and adds a marker naming the items: `reverify_mismatch=` (the recorded result no longer reproduces – the defect is back), `reverify_invalid_evidence=` (exit 2), `reverify_timeout=` (inconclusive), `reverify_refused=` (exit 4/5, e.g. lock busy) and `reverify_skipped_budget=`.

If a sub-check fails, the line instead carries `status=degraded` `reason=<list>` `failed=<list>` (reasons such as `db_missing`, `<surface>_rc<N>`) and a WARN is printed; the JSON body then holds only

the surfaces that succeeded. A missing/unreadable docs/workable_items.db is detected before any sqlite3 call (so a missing file is never silently created as an empty DB) and recorded as db_missing. If redaction itself fails the line falls back to status=degraded reason=redaction_failed.

In scripts/pre_build_verification.sh (stage [59/59] CM-ZERO-SHORTCOMINGS-STANDING) a status=ok run counts as a PASS; a status=degraded run, a non-zero exit, or a failure to create the stage's temp files each print a WARN line and are **not** counted as a PASS. The stage never calls fail() and never changes the sweep's exit status.

Run id format: YYYYMMDDTHHMMSSZ-pid<PID>.

Environment overrides

Variable	Default	Effect
WORKABLE_ITEMS_DB_OVERRIDE	docs/workable_items.db	Backlog reopens path.
GATE_LEDGER_SCRIPT_OVERRIDE	constitution/scripts/gates/cm_gate_ledger_ratchet.sh	Gate-keeper for the unimp count.
AUDIT_QA_ROOT	docs/qa	Root source <item-closure
AUDIT_STANDING_LOG_DIR	docs/qa/zero_shortcomings_audit	Directory standing (create)
AUDIT_INCIDENT_LOG_DIR	<AUDIT_QA_ROOT>/zero_shortcomings_audit	Directory closure incidents
AUDIT_VERIFY_COMMAND_TIMEOUT	unset (no bound)	Second command exceed
AUDIT_VERIFY_LOCK_FILE	\${TMPDIR:-/tmp}/zero_shortcomings_audit_verify_<cksum>.lock	FR-011
AUDIT_VERIFY_LOCK_WAIT	60	Second FR-011 refusing

Variable	Default	Effect
AUDIT_REVERIFY_DEFAULT	2	Closed check 1 -- reve given.
AUDIT_REVERIFY_ITEM_TIMEOUT	150	Per-ite timeou check.
AUDIT_REVERIFY_BUDGET	210	Re-ver in star item s timeou

Redaction (Constitution Principle III / §11.4.10)

Every value printed on a mismatch, the recorded command's stderr, every command quoted in a corruption incident, and every standing-check log line pass through `audit_redact_before_write`. Values are replaced by `<redacted-per-§11.4.10>`; names stay loggable. Comparison itself uses the raw values; only what is printed or written is redacted. Keywords (case-insensitive, matched anywhere in a name, e.g. `RUTRACKER_PASSWORD`): `password`, `passwd`, `secret`, `api_key` (also `api-key/` `apikey`), `access_token`, `auth_token`, `client_secret`, plus the project extension `key`, `token`, `cookies`.

The rules run in order, one line at a time:

1. `*_COOKIES=` / `Cookie:` – everything to the end of the line (cookie strings contain spaces).
2. `Authorization:` – everything to the end of the line.
3. URL userinfo `scheme://user:PASSWORD@host` – the password (the user stays); a token-only `scheme://TOKEN@host` (GitHub PAT clone form) – the whole token; and `curl -u user:PASSWORD` – the password.
4. `Bearer <token>` / `Basic <token>` – the token.
5. `<keyword>[" " ]?<sep>"double-quoted value"` – the whole quoted value (JSON style included).
6. The same with single quotes.
7. `<keyword>[" " ]?<sep>value` – the unquoted token (`sep = , =`, the fat arrow `=>`, or the URL-encoded `%3D/%3A`; spaces allowed around it).
8. A space-separated CLI flag `--<...keyword> value`.

9. (applied right after rule 3) A password glued to a mysql-family -p flag (mysql -uroot -pSECRET, also mysqldump, mysqladmin, mysqlimport, mysqlshow, mariadb, mariadb-dump) – case-sensitive, so the port flag -P3306 is untouched, and only on a line that runs such a client, so mkdir -p dir or git log -p stay intact. A bare mysql -p (interactive prompt) has no value and is left alone.

Known limits: an escaped quote inside a quoted value ("a\"b") ends the match early; a secret split across lines, a secret with no recognisable keyword before it (a bare token or base64 blob), and a value introduced by any other separator are **not** redacted. Bare key/token/bearer favour recall, so some non-secret text is over-redacted by design.

Inputs: docs/workable_items.db (read-only), constitution/scripts/gates/ (read-only, via cm_gate_ledger_ratchet.sh), docs/QA_DISCOVERY_LEDGER.md (read-only), docs/qa/<item-id>/closure_evidence_*.md (read; verify-closure).

Outputs: A human-readable or --json report; standing-check appends one line to <AUDIT_STANDING_LOG_DIR>/<run-id>.log; verify-closure appends to <AUDIT_QA_ROOT>/zero_shortcomings_audit/<run-id>.log (or \$AUDIT_INCIDENT_LOG_DIR) only when the corruption guard reverted something.

Side-effects: None in enumerate. standing-check writes its log line and, through its verify-closure re-runs, executes recorded commands (never reopening anything). verify-closure executes the recorded command (bash -c, fresh process, cwd = repo root), may revert corrupted tracked evidence files, and with --reopen-on-mismatch may call the workable-items CLI to reopen an item.

Dependencies: sqlite3, git, timeout, nice (ionice and flock optional, their absence is stated at run time), constitution/scripts/workable-items/workable-items, constitution/scripts/gates/cm_gate_ledger_ratchet.sh, and scripts/lib/audit_*.sh.

Cross-references: specs/003-zero-shortcomings-audit/{spec,plan,research,data-model}.md.

Last verified: 2026-09-26 (against the script source and the tests/audit/ suite; the --help text itself was run).